Decision Table Testing

Black Box Testing untuk Business Rules dan Kombinasi Kondisi

CONDITIONS - RULES - ACTIONS - TEST CASES

Dr. Sofiyanti Indriasari, S.Kom, M.Kom

Konsep Decision Table

Decision Table = representasi sistematis hubungan antara kombinasi kondisi dan tindakan/hasil yang diharapkan.

- Termasuk teknik Black Box Testing yang berfokus pada logika keputusan dan business rules.
- Digunakan ketika output sistem ditentukan oleh kombinasi beberapa kondisi, bukan hanya satu input.
- Setiap rule pada tabel menunjukkan satu kombinasi kondisi dan action yang seharusnya dilakukan sistem.

Mengapa Decision Table Dibutuhkan?

- Requirement sering berbentuk aturan IF THEN dengan beberapa kondisi yang saling berinteraksi.
- Menguji setiap input secara terpisah dapat melewatkan kesalahan yang hanya muncul pada kombinasi tertentu.
- Decision Table membantu memastikan kombinasi penting diperiksa secara sistematis dan dapat ditelusuri ke test case.

Pertanyaan utama:
“Jika beberapa kondisi terjadi bersamaan, apa yang seharusnya dilakukan sistem?”

Kapan Decision Table Digunakan?

- Requirement memiliki banyak aturan IF THEN / IF ELSE.
- Keputusan sistem bergantung pada dua atau lebih kondisi.
- Ada kombinasi input yang menghasilkan action berbeda.
- Business rules harus diuji secara lengkap dan konsisten.
- Contoh:

  diskon, approval, eligibility, registrasi, akses, pembayaran, dan workflow.

Konsep Dasar: Condition, Action, dan Rule

Elemen	Makna	Contoh
Condition	Faktor/input yang memengaruhi keputusan	Pelanggan member?
Action	Respons/hasil yang dilakukan sistem	Berikan diskon 20%
Rule	Satu kombinasi condition beserta actionnya	Member=Y & transaksi≥500k=Y maka diskon 20%

Satu kolom Decision Table = satu RULE

Anatomy of a Decision Table

	Rule 1	Rule 2	Rule 3
CONDITIONS			
Condition 1	Y	Y	N
Condition 2	Y	N	–
ACTIONS			
Action 1	v		
Action 2		v	v

- Condition Stub: daftar kondisi.
- Condition Entry: nilai Y/N/–.
- Action Stub: daftar tindakan.
- Action Entry: v = action dijalankan.
- Rule: satu kolom kombinasi.

Bagaimana Membaca Decision Table?

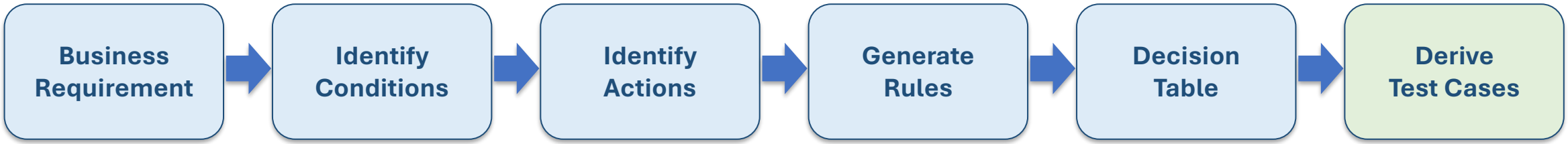
Baca secara vertikal: CONDITIONS ➔ ACTIONS

	R1	R2
Member?	Y	N
Transaksi ≥ Rp500.000?	Y	Y
Diskon 20%	v	
Diskon 5%		v

R1: Jika pelanggan member DAN transaksi ≥ Rp500.000 ➔ diskon 20%.

R2: Jika bukan member DAN transaksi ≥ Rp500.000 ➔ diskon 5%.

Alur Penyusunan Decision Table



Mulai dari requirement, bukan dari tabel.

Contoh Dasar: Sistem Diskon E-Commerce

Jika pelanggan member dan transaksi ≥ Rp500.000 ➔ diskon 20%.

Jika pelanggan member dan transaksi < Rp500.000 ➔ diskon 10%.

Jika pelanggan bukan member dan transaksi ≥ Rp500.000 ➔ diskon 5%.

Selain itu ➔ tidak memperoleh diskon.

Tujuan: menguji seluruh kombinasi status member dan nilai transaksi.

Step 1–2: Identify Conditions & Actions

Conditions	Actions
C1. Pelanggan member?	A1. Diskon 20%
C2. Transaksi ≥ Rp500.000?	A2. Diskon 10%
	A3. Diskon 5%
	A4. Tidak ada diskon

Conditions menjelaskan keadaan. Actions menjelaskan respons sistem.

Step 3: Determine Possible Combinations

Untuk n kondisi Boolean (Y/N), jumlah maksimum kombinasi = 2^n

Jumlah Condition	Maksimum Rules
1	2
2	4
3	8
4	16

Kasus diskon: 2 conditions ➔ 4 possible combinations

Step 4: Construct the Decision Table

	R1	R2	R3	R4
CONDITIONS				
Member?	Y	Y	N	N
Transaksi ≥ Rp500.000?	Y	N	Y	N
ACTIONS				
Diskon 20%	v			
Diskon 10%		v		
Diskon 5%			v	
Tidak ada diskon				v

Membaca Setiap Rule

R1: Member=Y, transaksi ≥500k=Y ➔ diskon 20%.

R2: Member=Y, transaksi ≥500k=N ➔ diskon 10%.

R3: Member=N, transaksi ≥500k=Y ➔ diskon 5%.

R4: Member=N, transaksi ≥500k=N ➔ tidak ada diskon.

Setiap rule harus konsisten dengan business requirement.

Deriving Test Cases dari Decision Table

TC	Member	Nilai Transaksi	Expected Result
TC01	Ya	Rp600.000	Diskon 20%
TC02	Ya	Rp300.000	Diskon 10%
TC03	Tidak	Rp600.000	Diskon 5%
TC04	Tidak	Rp300.000	Tidak ada diskon

Prinsip: setiap feasible rule direpresentasikan oleh minimal satu test case.

Decision Table dan Pemilihan Test Data

- Decision Table menentukan kombinasi kondisi yang harus diuji.
- EP/BVA dapat membantu memilih nilai konkret untuk merealisasikan condition pada rule.
- Contoh: condition ‘Transaksi ≥ Rp500.000 = Y’ dapat diuji dengan nilai batas Rp500.000 menggunakan BVA.

Decision Table ➔ WHAT combination?
EP/BVA ➔ WHICH test data?

Multiple Conditions: Mengapa Tabel Bisa Membesar?

3 conditions Boolean → 2^3 = 8 combinations

- Semakin banyak conditions, semakin banyak kombinasi yang mungkin.
- Tidak semua kombinasi selalu relevan atau perlu dipertahankan sebagai rule terpisah.
- Di sinilah konsep Don't Care dan simplification menjadi penting.

Combinatorial Explosion

Contoh Lanjutan: Registrasi Layanan Premium

C1. Data identitas lengkap?

C2. Pembayaran berhasil?

C3. Kode promo valid?

Jika data tidak lengkap ➔ registrasi ditolak.

Jika data lengkap tetapi pembayaran gagal ➔ registrasi ditolak.

Jika data lengkap + pembayaran berhasil + promo valid ➔ premium aktif dengan diskon.

Jika data lengkap + pembayaran berhasil + promo tidak valid ➔ premium aktif tanpa diskon.

Complete Decision Table: 3 Conditions, 8 Rules

	R1	R2	R3	R4	R5	R6	R7	R8
Data lengkap?	Y	Y	Y	Y	N	N	N	N
Pembayaran berhasil?	Y	Y	N	N	Y	Y	N	N
Promo valid?	Y	N	Y	N	Y	N	Y	N
Registrasi ditolak			v	v	v	v	v	v
Premium tanpa diskon		v						
Premium + diskon	v							

Pertanyaan: apakah nilai Promo masih penting ketika data tidak lengkap atau pembayaran gagal?

Konsep Don't Care (–)

“–” = nilai condition tidak memengaruhi action pada rule tersebut.

- Jika pembayaran gagal, registrasi tetap ditolak baik promo valid maupun tidak.
- Jika data identitas tidak lengkap, status pembayaran dan promo tidak mengubah keputusan penolakan.
- Don't Care membantu menggabungkan rules yang sebenarnya ekuivalen.

Y / N = relevan | – = tidak relevan untuk keputusan tersebut

Simplification: Dari 8 Rules Menjadi Effective Rules

Rule	Data Lengkap	Pembayaran Berhasil	Promo Valid	Result
R1	N	–	–	Registrasi ditolak
R2	Y	N	–	Registrasi ditolak
R3	Y	Y	N	Premium tanpa diskon
R4	Y	Y	Y	Premium + diskon

8 combinations ➔ identify irrelevant conditions ➔ Don't Care ➔ 4 effective rules

Prinsip Simplification

- Rule boleh digabung jika perbedaan nilai suatu condition tidak mengubah action yang diharapkan.
- Jangan menghapus rule hanya untuk memperkecil tabel.
- Prioritas business rule harus tetap terjaga.
- Setelah simplification, pastikan semua kombinasi awal tetap tercakup oleh effective rules.

Simplification ≠ mengurangi coverage

Derive Test Cases dari Simplified Table

TC	Data Lengkap	Pembayaran	Promo	Expected Result
TC01	Tidak	Apa pun	Apa pun	Registrasi ditolak
TC02	Ya	Gagal	Apa pun	Registrasi ditolak
TC03	Ya	Berhasil	Tidak valid	Premium tanpa diskon
TC04	Ya	Berhasil	Valid	Premium + diskon

Concrete test data dipilih saat test case ditulis dan dieksekusi.

Template Test Case untuk Eksekusi

ID	Pre-condition	Steps	Expected Output	Actual Output	Status
TCxx	Kondisi awal sistem	Langkah uji & data input	Hasil yang diharapkan	Diisi saat eksekusi	PASS/FAIL

- Rule pada Decision Table menjadi dasar logika test case.
- Actual Output dan Status diisi setelah pengujian dijalankan.
- Dokumentasikan bukti bila diperlukan (screenshot/log).

Decision Table vs EP & BVA

Teknik	Fokus Utama	Cocok untuk
Equivalence Partitioning	Kelas input yang ekuivalen	Domain input valid/invalid
Boundary Value Analysis	Nilai pada/sekitar batas	Input numerik, range, panjang
Decision Table	Kombinasi kondisi & business rules	Keputusan dengan banyak kondisi

Teknik dapat dikombinasikan untuk meningkatkan coverage.

Kelebihan dan Keterbatasan

Kelebihan	Keterbatasan
Sistematis untuk business rules	Jumlah rules cepat membesar
Menangkap kombinasi kondisi	Membutuhkan requirement yang jelas
Mudah ditelusuri ke test case	Tabel kompleks bila conditions terlalu banyak
Mendukung review konsistensi aturan	Perlu simplification yang hati-hati

Latihan

Kasus: Persetujuan Pesanan COD

- Condition:
  - alamat terjangkau?
  - nilai pesanan ≤ batas COD?
  - pelanggan tidak diblokir?
- Tentukan actions yang mungkin dilakukan sistem.
- Buat complete decision table (3 conditions ➔ 8 combinations).
- Identifikasi condition yang dapat menjadi Don't Care.
- Sederhanakan tabel tanpa kehilangan coverage.
- Turunkan effective rules menjadi test cases.

Praktikum: Terapkan pada Proyek

1. Pilih satu fitur proyek yang memiliki ≥2 business conditions.
2. Tuliskan business rules secara eksplisit.
3. Identifikasi Conditions dan Actions.
4. Buat Complete Decision Table.
5. Tandai impossible/irrelevant combinations jika ada.
6. Gunakan Don't Care dan lakukan simplification bila memungkinkan.
7. Derive test cases dari effective rules.
8. Eksekusi dan isi Actual Output serta PASS/FAIL.

Output Praktikum

Output	Isi
1. Business Rules	Aturan fitur yang diuji
2. Conditions & Actions	Daftar kondisi dan respons sistem
3. Complete Decision Table	Seluruh kombinasi awal
4. Simplified Decision Table	Effective rules + Don't Care bila relevan
5. Test Cases	ID, pre-condition, steps, expected output
6. Execution Result	Actual output + PASS/FAIL

Kesimpulan

Decision Table memetakan kombinasi conditions ke actions berdasarkan business rules.

Setiap kolom merepresentasikan satu rule yang dapat diturunkan menjadi test case.

Don't Care membantu menunjukkan condition yang tidak relevan terhadap suatu keputusan.

Simplification mengurangi redundansi tanpa mengurangi coverage.

Decision Table dapat dikombinasikan dengan EP/BVA untuk memilih test data yang lebih efektif.

Business Rules ➔ Decision Table ➔ Effective Rules ➔ Test Cases